The Journal of Open Source Software

# MetDataPy: A Source-Agnostic Toolkit for Meteorological Time-Series Data Preparation and Quality Control

**Kyriakos Kartas** [1]

**1** Independent Researcher, Greece

**DOI:** 10.xxxxxx/draft

**Software**

- Review
- Repository
- Archive

**Editor:**

**Submitted:** 25 October 2025
**Published:** unpublished

**License**
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

## Summary

MetDataPy is a Python package designed to streamline the ingestion, quality control, and preparation of meteorological time-series data for machine learning applications. Weather data from diverse sources—such as weather stations, citizen science networks, and IoT sensors—typically arrive in heterogeneous formats with varying units, inconsistent timestamps, and quality issues. MetDataPy addresses these challenges by providing a unified canonical schema, automated column mapping with source detection, comprehensive quality control algorithms, and reproducible ML-ready data pipelines. The package enables researchers and practitioners to transform raw meteorological observations into clean, well-documented datasets suitable for forecasting, climate analysis, and agricultural modeling. Install via: `pip install metdatapy` (Python 3.9+, MIT). Repository: GitHub. Documentation: Read the Docs.

## Statement of Need

MetDataPy primarily serves atmospheric scientists, agricultural researchers, and machine learning practitioners working with weather station data and meteorological time series.

Machine learning applications in atmospheric sciences, agriculture, and renewable energy increasingly rely on high-quality meteorological time-series data (Rasp et al., 2020; Schultz et al., 2021). However, preparing such data remains time-consuming and error-prone due to several challenges: (1) heterogeneous data formats across providers (NOAA, Weathercloud, custom stations); (2) unit inconsistencies (Fahrenheit vs. Celsius, mph vs. m/s); (3) temporal issues including timezone handling, daylight saving transitions, and irregular sampling; (4) quality problems such as sensor drift, outliers, and missing values; and (5) ML-specific requirements like time-safe data splitting and feature scaling. A typical workflow might involve manually mapping dozens of column names, writing custom unit conversion functions, implementing ad-hoc quality checks, and carefully managing train-test splits to prevent temporal leakage—processes that are both error-prone and difficult to reproduce across research groups.

Existing tools address subsets of these challenges. `pandas` (McKinney, 2010) provides general-purpose time-series manipulation but lacks meteorology-specific quality control and domain knowledge. MetPy (May et al., 2022) offers comprehensive meteorological calculations but focuses primarily on thermodynamic computations and vertical profile analysis rather than data ingestion and ML preparation. `xarray` (Hoyer & Hamman, 2017) excels at multidimensional climate data but requires users to implement quality control manually and does not provide ML-oriented preprocessing. `tsfresh` (Christ et al., 2018) automates time-series feature extraction but does not address domain-specific meteorological derived variables, quality flagging protocols, or the unique challenges of weather data (e.g., wind direction circularity, accumulated precipitation rollovers).

MetDataPy fills this gap by providing an end-to-end workflow specifically designed for meteorological time-series data preparation. Its key contributions include:

1. **Source-agnostic ingestion**: Automatic column detection using heuristics (regex patterns, unit hints, statistical plausibility) with confidence scoring, enabling seamless integration of data from diverse providers without manual schema mapping. The detector assigns confidence scores based on name matching, unit inference, and statistical plausibility checks against meteorological bounds, with an interactive wizard for refinement (CLI: `mdp ingest detect`).

2. **Comprehensive quality control**: Meteorological QC algorithms including range checks against climatological bounds, spike detection using rolling Median Absolute Deviation (Leys et al., 2013) which is more robust than standard deviation, flatline detection via rolling variance to identify stuck sensors, and cross-variable consistency checks exploiting physical relationships (e.g., dew point  temperature, wind direction undefined when wind speed  0). All checks produce boolean flags preserving provenance without destructive deletion. QC parameters are configurable via YAML, with reports providing statistical summaries by variable and flag type.

3. **Derived meteorological metrics**: Scientifically validated calculations of dew point using the Magnus-Tetens approximation (Lawrence, 2005), vapor pressure deficit following FAO-56 guidelines (Allen et al., 1998), heat index based on Steadman's physiological model (Steadman, 1979) with the Rothfusz regression approximation (Rothfusz, 1990), and wind chill following the 2001 North American wind chill equivalent temperature chart (Osczevski & Bluestein, 2005). All formulations include complete references to peer-reviewed literature and specify valid ranges and assumptions. These derived variables are crucial for agricultural decision support systems, outdoor worker safety applications, and energy demand forecasting.

4. **ML-ready preparation**: Time-safe data splitting preventing temporal leakage, supervised learning table generation with configurable lags and forecast horizons, and reproducible scaling with parameter serialization. The package supports standard, min-max, and robust scaling with parameters serialized to JSON for exact reproducibility. Calendar features (hour, day-of-week, month, cyclical encodings) are automatically generated, and external covariates can be aligned by timestamp.

5. **Reproducibility infrastructure**: CF-compliant NetCDF export ("Conventions=CF-1.8") (Eaton et al., 2020) with comprehensive metadata and proper QC flag handling. Manifest generation tracks all pipeline steps with timestamps, software versions, parameters, and a deterministic pipeline hash. Pydantic-validated schemas ensure type safety and enable manifest comparison for reproducibility verification. Command-line tools (`mdp manifest validate`, `show`, `compare`) facilitate auditing and debugging.

# Implementation and Design

The package is structured around a core `WeatherSet` class providing a fluent API for chaining operations while maintaining data provenance. The canonical schema defines nine core meteorological variables with standardized names and SI-derived units (e.g., `temp_c`, `rh_pct`, `pres_hpa`, `wspd_ms`, `wdir_deg`, `rain_mm`). Timezone handling ensures all timestamps are converted to UTC, eliminating ambiguity from daylight saving transitions. Special handling detects and corrects accumulated rainfall sensor rollovers. Resampling operations intelligently aggregate variables (mean for intensive quantities, sum for extensive) while conservatively propagating quality flags using logical OR.

The implementation emphasizes computational efficiency through vectorized pandas and NumPy operations. On one year of 10-minute station data (~52,560 rows), core operations (QC, derived metrics, resampling, calendar features) typically exceed 100,000 rows/s on consumer

hardware (Python 3.11, NumPy 2.3, pandas 2.2). Reproducible benchmarks are provided in `benchmarks/benchmark_performance.py`.

The command-line interface (`mdp`) provides composable commands for detection, ingestion, quality control, and export, enabling integration into automated pipelines. The Python API allows fine-grained control for Jupyter notebooks and custom scripts. Comprehensive documentation includes API references, tutorials, and a publication-quality Jupyter notebook with visualizations.

# Example

```python
import pandas as pd
from metdatapy.mapper import Mapper
from metdatapy.core import WeatherSet

df = pd.read_csv("data/sample_weather_2024.csv")
mapping = Mapper().detect(df.head(200))
ws = WeatherSet.from_mapping(df, mapping).to_utc().normalize_units(mapping)
ws = ws.qc_range().qc_spike().qc_flatline().qc_consistency()
clean = ws.derive(["dew_point", "vpd"]).resample("1H").to_dataframe()
```

All analyses and benchmarks reported in this paper are fully reproducible using the provided example notebooks (`examples/metdatapy_tutorial.ipynb`), benchmark scripts (`benchmarks/benchmark_performance.py`), sample data (`data/sample_weather_2024.csv` - a full year of synthetic 10-minute weather station data with realistic patterns), and integration tests in the repository. To reproduce benchmarks: `python benchmarks/benchmark_performance.py`.

# Use Cases and Applications

MetDataPy supports diverse applications across atmospheric and agricultural sciences. In **precision agriculture**, it processes weather station data to compute growing degree days and evapotranspiration for irrigation scheduling. For **renewable energy forecasting**, it prepares historical weather data with appropriate lags and derived features for training solar and wind power prediction models. In **urban climate studies**, it harmonizes data from heterogeneous sensor networks, applying consistent quality control before spatial interpolation. **Climate model validation** workflows use MetDataPy to prepare observational benchmarks with uncertainty quantification via QC flags, enabling comparisons with model outputs in CF-compliant NetCDF format.

The package prioritizes scientific correctness, reproducibility, and ease of use, lowering the barrier for researchers to adopt best practices. By providing well-tested implementations with complete literature references, MetDataPy reduces duplication of effort and potential errors in custom preprocessing code.

# Acknowledgements

The development of MetDataPy was inspired by challenges encountered in real-world meteorological data processing workflows and benefited from the extensive Python scientific computing ecosystem, including NumPy (Harris et al., 2020), pandas (McKinney, 2010), and scikit-learn (Pedregosa et al., 2011). The project welcomes community contributions following the guidelines in CONTRIBUTING.md, and adheres to the Contributor Covenant Code of Conduct to foster an inclusive and collaborative development environment.

DRAFT

# References


Allen, R. G., Pereira, L. S., Raes, D., & Smith, M. (1998). *Crop evapotranspiration-guidelines for computing crop water requirements*. FAO Irrigation; drainage paper 56; Food; Agriculture Organization of the United Nations. http://www.fao.org/3/x0490e/x0490e00.htm

Christ, M., Braun, N., Neuffer, J., & Kempa-Liehr, A. W. (2018). Time series feature extraction on basis of scalable hypothesis tests (tsfresh–a python package). *Neurocomputing*, *307*, 72–77. https://doi.org/10.1016/j.neucom.2018.03.067

Eaton, B., Gregory, J., Drach, B., Taylor, K., Hankin, S., Blower, J., Caron, J., Signell, R., Bentley, P., Rappa, G., & others. (2020). *NetCDF climate and forecast (CF) metadata conventions, version 1.8*. http://cfconventions.org/Data/cf-conventions/cf-conventions-1.8/cf-conventions.html.

Harris, C. R., Millman, K. J., Van Der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hoyer, S., & Hamman, J. (2017). Xarray: N-d labeled arrays and datasets in python. *Journal of Open Research Software*, *5*(1), 10. https://doi.org/10.5334/jors.148

Lawrence, M. G. (2005). The relationship between relative humidity and the dewpoint temperature in moist air: A simple conversion and applications. *Bulletin of the American Meteorological Society*, *86*(2), 225–233. https://doi.org/10.1175/BAMS-86-2-225

Leys, C., Ley, C., Klein, O., Bernard, P., & Licata, L. (2013). Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median. *Journal of Experimental Social Psychology*, *49*(4), 764–766. https://doi.org/10.1016/j.jesp.2013.03.013

May, R. M., Arms, S. C., Marsh, P., Bruning, E., Leeman, J. R., Goebbert, K., Thielen, J. E., Bruick, Z. S., & Camron, M. D. (2022). MetPy: A meteorological python library for data analysis and visualization. *Bulletin of the American Meteorological Society*, *103*(10), E2273–E2284. https://doi.org/10.1175/BAMS-D-21-0125.1

McKinney, W. (2010). Data structures for statistical computing in python. *Proceedings of the 9th Python in Science Conference*, *445*, 51–56. https://doi.org/10.25080/Majora-92bf1922-00a

Osczevski, R., & Bluestein, M. (2005). The new wind chill equivalent temperature chart. *Bulletin of the American Meteorological Society*, *86*(10), 1453–1458. https://doi.org/10.1175/BAMS-86-10-1453

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., & others. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, *12*, 2825–2830. http://jmlr.org/papers/v12/pedregosa11a.html

Rasp, S., Dueben, P. D., Scher, S., Weyn, J. A., Mouatadid, S., & Thuerey, N. (2020). WeatherBench: A benchmark data set for data-driven weather forecasting. *Journal of Advances in Modeling Earth Systems*, *12*(11), e2020MS002203. https://doi.org/10.1029/2020MS002203

Rothfusz, L. P. (1990). *The heat index equation (or, more than you ever wanted to know about heat index)*. National Weather Service Technical Attachment SR 90-23; National Oceanic; Atmospheric Administration. https://www.weather.gov/media/ffc/ta_htindx.PDF

Schultz, M. G., Betancourt, C., Gong, B., Kleinert, F., Langguth, M., Leufen, L. H., Mozaffari, A., & Stadtler, S. (2021). Can deep learning beat numerical weather prediction? *Philosophical Transactions of the Royal Society A*, *379*(2194), 20200097. https:


DRAFT

//doi.org/10.1098/rsta.2020.0097

Steadman, R. G. (1979). The assessment of sultriness. Part i: A temperature-humidity index based on human physiology and clothing science. *Journal of Applied Meteorology and Climatology*, *18*(7), 861–873. https://doi.org/10.1175/1520-0450(1979)018%3C0861:TAOSPI%3E2.0.CO;2

DRAFT